

Programmazione Web e Mobile

Appunti del corso

2025/2026

Introduzione

Cos'è il World Wide Web? è un servizio di Internet che permette di navigare e usufruire di contenuti. Agisce connettendo i vari nodi della rete (come computer e dispositivi) e si basa su un sistema ipertestuale, ovvero costituito da testi che contengono rimandi diretti ad altri testi.

- HTML: il linguaggio di formattazione (markup) con cui sono scritte le pagine web.
- HTTP: il protocollo di rete che opera a livello applicazione della pila ISO/OSI.
- URL: lo schema di identificazione dei contenuti e dei servizi presenti sul Web.

HTML - Hyper Text Markup Language

Sintassi stabilita dal World Wide Web Consortium (W3C) Definisce la struttura delle pagine web HTML 5.3 rilasciato il 28 gennaio 2021.

Browser: Applicazione per l'acquisizione, la presentazione e la navigazione di risorse sul Web (pagine HTML). Implementa funzionalità di client per il protocollo HTTP, che regola il download delle risorse dai server web a partire dal loro indirizzo URL.

HTTP - Hyper Text Transfer Protocol

Transmission Control Protocol (TCP):

connette due hosts assicura che i pacchetti arrivino nello stesso ordine in cui vengono inviati

Transport Layer Security (TLS):

rende privata e sicura la comunicazione tra due hosts, impedendo intrusioni

Funzionalità: Cache, CORS, Autenticazione, Proxy, Sessioni

HTTP è stateless, ovvero non c'è collegamento tra due richieste consecutive. Dunque si usano i cookies.

Richiesta → inviata dal client (browser) al server web

- Method: definisce l'operazione da eseguire
- Path: la posizione in rete della risorsa da recuperare
- Header: informazioni aggiuntive da inoltrare con la richiesta
- Body: (opzionale) contiene una risorsa da inviare

```
GET / HTTP/1.1
Host: developer.mozilla.org
Accept-Language: fr
```

Risposta → inviata dal server web al client (browser)

- Status code/message: indica se la richiesta è riuscita e perché
- Body: (opzionale) contiene la risorsa richiesta

```
HTTP/1.1 200 OK
date: Tue, 18 Jun 2024 10:03:55 GMT
cache-control: public, max-age=3600
content-type: text/html
```

- A partire da HTTP/2 i messaggi in incapsulati in frames binari non intellegibili
- HTTP/3 funziona sul protocollo di rete QUIC che a sua volta utilizza UDP come protocollo di trasporto

URL (Uniform Resource Locator)

- Identifica univocamente l'indirizzo di una risorsa in rete
- Il numero di porta è opzionale e alcuni protocolli ne hanno uno riservato (es. 80 per HTTP)
- La risoluzione dell'URL in indirizzo IP avviene tramite DNS, una sorta di rubrica di Internet

Web 1.0

Contenuti statici, solo lettura

Creati esclusivamente da programmatori/aziende

HTML semplice, nessuna interazione utente

Web 2.0

Contenuti dinamici e in tempo reale (AJAX)

Generati dagli utenti (es. social media, YouTube, ...)

Interattività e collaborazione tra persone

Web 3.0

Semantico: le macchine capiscono il significato dei contenuti

AI e Machine Learning per personalizzazione

Integrazione profonda tra piattaforme e servizi

Componenti di una pagina

1. Struttura e contenuti (HTML)
2. Rappresentazione visiva (CSS)
3. Funzionalità e interazione (JavaScript)

N.B. disaccoppiare la struttura logica di una pagina web e la sua rappresentazione.

1. HTML

Mantenuto da W3C e WHATWG. Tag semantici (article, section, ...), multimedia nativo, mobile-first.

Tag Contenitori: Racchiudono contenuti

```
<div></div>
<p></p>
<section></section>
```

Tag Blocco: Larghezza piena, iniziano su nuova riga

```
<div>
<h1> - <h6>
<p>
```

Tag Vuoti: Nessun tag chiusura

```
<img />
<input />
<hr />
```

Tag Inline: Larghezza contenuto, stessa riga

```
<span></span>
<a></a>
<img />
```

Struttura base

```
<!DOCTYPE html> <!-- indica che il documento è scritto in HTML5 -->
<html lang="it"> <!-- inizio del documento -->
<head> <!-- metadati e informazioni invisibili -->
  <meta charset="utf-8"> <!-- codifica dei caratteri -->
  <title>Titolo pagina</title> <!-- mostrato nella scheda del browser -->
</head>
<body> <!-- contiene tutti gli elementi visibili -->
  Contenuto visibile
</body>
</html>
```

Elementi e strutture

Liste

- Ordinate: con
- Non ordinate: con
- Descrizione: <dl> con <dt>, <dd>

Citazioni

- <q>: inline, aggiunge virgolette automaticamente
- <blockquote>: blocco, indentato

Tabelle

```
<table> <!-- contenitore -->
  <tr> <!-- riga -->
    <th>Nome</th> <!-- intestazione -->
    <th>Età</th>
  </tr>
  <tr>
    <td>Marco</td> <!-- dati -->
    <td>30</td>
  </tr>
</table>
```

colspan (colonne): cella occupa più colonne

```
<td colspan="2">0ccupa 2 col</td>
```

rowspan (righe): cella occupa più righe

```
<td rowspan="2">0ccupa 2 rig</td>
```

Form

```
<form action="/submit" method="POST">
  <label for="nome">Nome:</label> <!-- Etichetta associata all'input via for-->
  <input type="text" id="nome" name="nome" />

  <label for="categoria">Categoria:</label>
  <select id="categoria" name="categoria"> <!-- Dropdown menu -->
    <option value="tech">Tecnologia</option> <!-- Valore inviato al server -->
    <option value="sport">Sport</option>
    <option value="musica">Musica</option>
  </select>

  <label for="messaggio">Messaggio:</label>
  <textarea id="messaggio" name="messaggio">
    <!-- Testo multiriga-->
  </textarea>

  <button type="submit">Invia</button> <!-- Pulsante submit/reset/custom -->
</form>
```

Attributi <form>:

- action: dove inviare dati
- method: GET (URL) o POST (body)

Altri elementi form:

- <fieldset> + <legend>: raggruppa campi con titolo

Campo <input type="text" /> con vari tipi:

- text, email, password: campi testo
- number, date, range: input numerici/date
- checkbox, radio: selezioni multiple/singola
- file, hidden: file, dati nascosti

Multimedia

Video

```
<video width="320" height="240" controls>
  <source src="video.mp4" type="video/mp4">
  Browser non supporta video
</video>
```

Attributi: controls, autoplay, loop, muted

Audio

```
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
  Browser non supporta audio
</audio>
```

Attributi: controls, autoplay, loop, muted

Link e Anchor

```
<a href="https://example.com" target="_blank">Link esterno</a>
<a href="#sezione1">Vai a sezione 1</a>
<section id="sezione1">Sezione 1</section>
```

- href: URL o id elemento
- target="_blank": apri in nuova scheda

Tag semantici

Migliorano struttura e SEO:

- <header>: intestazione pagina/sezione
- <nav>: navigazione
- <main>: contenuto principale
- <article>: contenuto indipendente (post, articolo)
- <section>: sezione tematica
- <aside>: contenuti secondari
- <footer>: piè di pagina

Attributi HTML

- id: identificatore univoco elemento
- class: classe CSS (multiple)
- alt: descrizione immagini (accessibilità, SEO)
- title: tooltip al hover
- data-*: dati personalizzati
- role: ruolo elemento (accessibilità ARIA)
- aria-*: attributi accessibilità ARIA

■ Tag semantici = struttura logica + accessibilità + SEO. Fondamentali per pagine ben organizzate.

2. CSS (Cascading Style Sheets)

Migliora UX. Riduce tempo e sforzo design. Adatta layout a diverse dimensioni.

Collegamento: `<link rel="stylesheet" href="style.css">` in `<head>`

Proprietà comuni

Font e testo

- `font-family`: famiglia font
- `font-size`: grandezza
- `font-weight`: spessore (100-900, bold)
- `font-style`: italic, normal, oblique
- `text-align`: left, center, right, justify
- `text-decoration`: underline, overline, line-through
- `text-transform`: uppercase, lowercase, capitalize

Colori e sfondo

- `color`: colore testo
- `background-color`: colore sfondo
- `background-image`: immagine sfondo
- `background-size`: cover, contain
- `background-position`: posizione immagine
- `background-repeat`: repeat, no-repeat, repeat-x, repeat-y

- **Selettore**: identifica elementi.
- **Blocco dichiarazione**: {}
contiene proprietà e valori, separati da ;

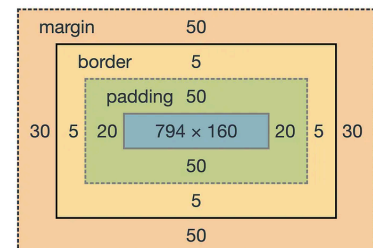
Selettori CSS

- `*`: universale
- `p`: di tipo
- `.classe`: di classe
- `#id`: per ID
- `p span`: discendente (span dentro p)
- `div > p`: figlio diretto
- `div + p`: sibling successivo
- `div ~ p`: sibling qualsiasi

Box Model

Ogni elemento: **content** → **padding** → **border** → **margin**

- `width/height`: dimensioni contenuto
- `padding`: spazio interno (da content a border)
- `border`: contorno, personalizzabile (larghezza, stile, colore)
- `margin`: spazio esterno (da border a altri elementi)
- `border-radius`: angoli arrotondati



Display

Proprietà display definisce il layout di un elemento.

- **Inline**: stessa riga, larghezza contenuto, no width/height
- **Block**: larghezza 100%, nuova riga, accetta width/height/margin/padding
- **Inline-block**: inline + proprietà blocco (width, height)
- **Flex**: layout flessibile. Allinea elementi lungo asse principale/trasversale.
 - `flex-direction`: row, column
 - `flex-wrap`: wrap, nowrap
 - `align-content`: spazio tra righe
 - `justify-content`: allineamento asse principale
 - `align-items`: allineamento asse trasversale
- **Grid**: layout a griglia righe×colonne.
 - `grid-template-columns`: numero e larghezza colonne
 - `grid-template-rows`: numero e altezza righe
 - `grid-gap`: spazio tra celle
 - `grid-column`: colonna inizio/fine
 - `grid-row`: riga inizio/fine

- **Table:** simula comportamento tabella HTML con CSS.
 - `display: table:` contenitore tabella
 - `display: table-row:` rappresenta riga tabella
 - `display: table-cell:` rappresenta cella tabella

Layout a colonne permette di distribuire il testo automaticamente in più colonne:

- `column-count:` numero fisso colonne
- `column-width:` larghezza minima colonne (adatta numero)
- `column-gap:` spazio tra colonne
- `column-rule:` bordo tra colonne (larghezza, stile, colore)

```
.articolo {
  column-count: 3;
  column-gap: 2em;
  column-rule: 1px solid #ccc;
}
```

Testo si distribuisce automaticamente su colonne (tipo rivista).

Posizionamento

Proprietà `position` determina il metodo di posizionamento di un elemento.

- `static` (default): flusso normale
- `relative:` relativo a posizione originale
- `absolute:` relativo a antenato posizionato, fuori flusso
- `fixed:` relativo finestra browser
- `sticky:` fisso quando scrolling supera soglia

Proprietà `offset`: `top`, `right`, `bottom`, `left`

Allineamento

Orizzontale:

- `Inline: text-align:` `center`
- `Block: margin:` `auto` con `width` definita
- `Flex: justify-content:` `center`
- `Grid: justify-items:` `center`

Verticale:

- `Inline: vertical-align:` `middle`
- `Block: display: flex; align-items:` `center`
- `Grid: align-items:` `center`

Float: posiziona elemento a sinistra/destra dentro contenitore, testo e altri elementi si avvolgono attorno:

- `float: left:` elemento a sinistra
- `float: right:` elemento a destra
- `clear:` impedisce wrapping (`left`, `right`, `both`)

■ *Float risalente a era pre-flexbox. Preferire flex/grid per layout moderni.*

Effetti CSS3

- **Box/Text-shadow:** ombra elemento/testo (`h-offset`, `v-offset`, `blur`, `spread`, `color`)
- **Gradient:** transizione colori fluide (`linear-gradient`, `radial-gradient`)
- **Transform:** trasformazioni 2D/3D (`translate()`, `rotate()`, `scale()`, `skew()`)
- **Transition:** anima cambi proprietà CSS. Proprietà:
 - `transition-property:` proprietà da animare
 - `transition-duration:` durata
 - `transition-timing-function:` curva accelerazione
 - `transition-delay:` ritardo
- **Animation:** animazioni complesse con `keyframes`. Proprietà:
 - `animation-name:` nome animazione
 - `animation-duration:` durata
 - `animation-timing-function:` curva
 - `animation-delay:` ritardo
 - `animation-iteration-count:` ripetizioni

- **:focus**: Pseudo-classe CSS applicata elemento quando riceve focus (click o Tab). Evidenzia elementi attivi
- Esempio: `button:focus { outline: 2px solid blue; }` evidenzia pulsante attivo con bordo blu.

Tag e :

- `list-style-type`: tipo marker (disc, circle, square, etc.)
- `list-style-image`: immagine marker
- `list-style-position`: inside, outside
- `list-style`: shorthand

Tag <q> (inline):

- `quotes`: personalizza virgolette automatiche

Esempio:

```
blockquote {
  margin-left: 40px;
  border-left: 4px solid #ccc;
  padding-left: 15px;
}
```

Tag <blockquote> (blocco):

- `margin-left`: rientro sinistro
- `border-left`: bordo sinistro per enfasi
- `padding-left`: spazio interno da bordo

Responsive Design

Approccio: adatta layout a diverse dimensioni schermo (desktop, tablet, smartphone).

Media query: applica CSS in base a condizioni

Condizioni:

- `max-width`
- `min-width`
- `orientation` (portrait/landscape)
- `screen`
- `print`

```
@media (max-width: 768px) {
  /* stili tablet */
}

@media (max-width: 480px) {
  /* stili smartphone */
}
```

Best practices

- Unità relative: %, em, rem (non px fissi)
- Layout flex/grid per adattabilità
- Viewport meta tag:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Menu di navigazione

Elemento chiave sito web. Consente navigazione facile tra sezioni e pagine.

Tipi di menu:

- **Orizzontale**: barra top/bottom, link affiancati, ideale desktop
- **Verticale**: sidebar, link in colonna, navigazione strutturata
- **Dropdown**: sottomenu al hover, organizza categorie gerarchiche
- **Burger**: icon menu mobile, toggle per mostrare/nascondere link

Best practices:

- Indicare sezione attiva (colore/sottolineatura)
- Navigabile da tastiera (tab)
- Responsive: cambia layout per mobile
- Stile coerente resto sito
- Link distinguibili e ben spaziati

Validazione

Garantisce conformità standard W3C. Evita problemi:

- Visualizzazione scorretta e elementi mancanti
- Accessibilità compromessa (difficoltà per utenti con disabilità)
- SEO negativa (ranking inferiore, indicizzazione inefficiente)
- Manutenzione difficile e debugging complesso
- Compatibilità limitata con browser/standard nuovi
- Esperienza utente negativa (frustrazione)

3. Web Design

Progettazione grafica e UX efficace. Comunica identità, guida navigazione, crea emozione.

Font

Font leggibili migliorano UX. Scegli:

- Browser-safe: Arial, Verdana
- Online: Google Fonts

Differenzia: titolo, sottotitolo, corpo

Icone

- Semplificano navigazione, attirano attenzione.
- Mantieni stile coerente.

Librerie: Font Awesome, Material Icons.

Colore

Comunica emozioni, identità brand, dirige interazioni.

Modelli di colore

- **RGB:** Red, Green, Blue - display digitali
- **CMYK:** Cyan, Magenta, Yellow, Black - stampa
- **HSL:** tonalità (0-360°), saturazione (0-100%), luminosità (0-100%)

Colori esadecimali (Hex)

Notazione #RRGGBB. Tre coppie hex (00-FF) canali Red, Green, Blue.

Esempi: #000000 nero, #FFFFFF bianco, #FF0000 rosso.

■ *Color picker per ottenere valori hex da immagini/design.*

Psicologia colori

Blu: fiducia, professionalità
Verde: natura, benessere
Verde scuro: finanza
Rosso: passione, urgenza
Giallo: gioia, creatività
Arancione: energia, entusiasmo
Rosa: femminilità, autenticità
Viola: lusso, mistero, arte
Nero: eleganza, potenza
Bianco: purezza, semplicità

Ruota dei colori

Strumento visivo per selezione colori e comunicazione design:

- **Primari:** rosso, blu, giallo
- **Secondari:** verde, arancione, viola (mescolanza due primari)
- **Terziari:** mescolanza primario + secondario

Armonie cromatiche

- **Complementari:** opposti ruota, contrasto forte (blu, arancione)
- **Analoghi:** vicini, effetto naturale (verde, giallo-verde, giallo)
- **Triadici:** tre equidistanti, equilibrio vivace (rosso, blu, giallo)
- **Tetradici:** due coppie complementari, varietà ricchezza (rosso, verde, blu, arancione)

Palette monocromatiche

- Schema singolo colore: mantenendo hue costante saturazione e luminosità variabili (0-100%).
- Effetto armonioso, coerente.
- Utile per brand identità lineare.

Palette acromatiche

- Bianco, grigio, nero (nessun colore).
- Eleganza, professionalità, minimalismo.
- Spesso con colore accento.
- Ideale per design minimalista, tech, finanza.

Audience: Comprendi target audience. Esigenze, abitudini, aspettative reali determinano design efficace. Senza conoscenza audience, design rischia non rispondere necessità persone usando sito.

Composizione visiva

Sezione aurea nel web si dimensionano colonne e sezioni usando il rapporto 1.618:

1. Definire larghezza totale
2. Sidebar: larghezza / 1.618
3. Content principale: larghezza - sidebar

Crea proporzioni esteticamente armoniose e naturali.

Regola dei terzi (applicata) griglia 3×3 per posizionare elementi:

- Titoli nei terzi verticali (preferibilmente sinistra)
- Immagini nel terzo destro
- Call-to-action presso punti intersezione inferiori

Migliora bilanciamento e dinamica composizione.

Metafora visiva

Immagini e simboli comunicano più veloce di testo.

Usare metafore familiari:

- Cartella: documenti, raccolta
- Nuvola: storage, sincronizzazione
- Lucchetto: sicurezza, protezione
- Ingranaggio: impostazioni, configurazione
- Cestino: eliminazione

Riduce carico cognitivo utente.

Affordance nel design

L'elemento comunica l'azione intuitiva.

Il bottone deve sembrare cliccabile:

- Colore contrastante
- Forma distintiva (rettangolo, pill, icona)
- Padding visibile
- Profondità (ombra, bordo, gradiente)
- Cursor pointer al hover

Un design intuitivo non richiede spiegazioni.

Gerarchia visiva

Guidare utente verso informazioni importanti:

- **Dimensione:** elementi grandi attirano attenzione
- **Colore:** colori vivaci evidenziano info chiave
- **Posizione:** contenuti in alto/centro ricevono più attenzione
- **Contrasto:** differenza tonale enfatizza elementi
- **Spaziatura:** margini intorno a elemento isolano e enfatizzano

Creare percorso visivo logico per utente.

Errori da evitare

- **PDF online:** non responsivo, preferire HTML.
- **Link visitati (:visited) non differenziati:** utenti perdono traccia. Differenziare con colore.
- **Testo non scansionabile:** spezzare con heading, liste, paragrafi brevi.
- **Dimensioni fisse:** usare unità relative (% , em, rem) per responsività.
- **Elementi pseudo-pubblicità:** confondono utenti, ridurre credibilità. Mantener design pulito.
- **Ignorare standard W3C:** compromette accessibilità, SEO, compatibilità browser.

Best Practices

- Contrasto sufficiente per accessibilità e leggibilità
- Schema colori coerente su tutte pagine per rafforzare brand
- Colori complementari per enfasi elementi importanti
- Evitare colore come unico mezzo trasmettere informazioni
- Testare design con strumenti che simulano forme daltonismo
- Audience analysis prima di qualsiasi scelta design
- Applicare sezione aurea e rule of thirds per composizione armoniosa
- Semplificare con metafore visive per attenzione breve utenti

4. Scrivere per il Web

Leggibilità

Testi leggibili migliorano UX. Strategie:

- Titoli, sottotitoli, paragrafi brevi per scansione veloce
- Evidenziare parole chiave e concetti chiave
- Font non complessi, dimensioni adeguate (min 1em corpo)
- Colori con contrasto elevato tra testo e sfondo
- Spaziatura tra paragrafi evita confusione
- Audience analysis determina scelte design

Accessibilità

Consente navigazione a tutti utenti, inclusi disabili visivi/cognitivi.

Rendere sito accessibile a tutti:

- Contrasto elevato testo-sfondo
- Descrizioni immagini e contenuti multimediali (alt text)
- Interazione senza mouse (Tab, Enter)
- Dimensioni relative (% , em, rem) per ridimensionamento testo

Tag HTML

- `<label>`: associa etichette a input (attributo `for`)
- organizzano pagina: `<header>`, `<nav>`, `<main>`, `<footer>`

Attributi:

- `alt`: descrive immagini
- `tabindex`: ordine navigazione elementi interattivi
- `role`: specifica ruolo elemento (navigation, button, progressbar)
- `aria-*`: Standard W3C con attributi per migliorare accessibilità
 - Esempio: `aria-valuenow`, `aria-valuemin`, `aria-valuemax` per progressbar

Daltonismo (Color Blindness):

- Deuteranopia: difficoltà rosso-verde
- Protanopia: difficoltà rosso
- Tritanopia: difficoltà blu-giallo

Soluzioni:

- Combinazioni colori garantiscono distinzione chiara testo-sfondo
- Evitare colore come unico mezzo trasmettere informazioni
- Aggiungere icone/descrizioni testuali insieme colori
- Testare design con strumenti simulazione daltonismo

Tool online: Adobe Color Contrast Analyzer, Lighthouse per Chrome.

5. JavaScript

Linguaggio programmazione client-side. Aggiunge interattività a pagine web, modifica HTML/CSS in tempo reale, risponde a eventi utente.

Caratteristiche:

- Esecuzione nel browser (client-side)
- Modifica DOM, HTML, CSS dinamicamente
- Gestione eventi (click, input, scroll)
- Richieste asincrone (AJAX) senza ricaricare pagina

Dove inserire JavaScript

Nel tag `<head>`

- Esecuzione prima contenuto
- Script inizializzazione, librerie

Fine tag `<body>` (consigliato):

- Evita blocco rendering pagina
- DOM già caricato
- Script interagisce facilmente con elementi

In mezzo tag `<body>`

- Utile se dipende da parte specifica contenuto

File esterno:

- Separazione codice, mantenibilità
- Caching browser
- Riutilizzo script su più pagine

Variabili

Tre modi dichiarazione:

1. **var**: ambito globale o funzione (più vecchia, sconsigliata)
2. **let**: ambito blocco, ad esempio interno ciclo for (moderna, sicura)
3. **const**: non riassegnabile dopo dichiarazione

Funzioni

Eseguono blocchi codice, ritornano risultato. Tre tipi:

Funzioni dichiarate:

```
function somma(a, b) {  
  return a + b;  
}
```

Funzioni anonime:

```
let somma = function(a, b){  
  return a + b;  
};
```

Arrow function:

```
let somma = (a, b) => a + b;
```

Strutture di controllo

Gestiscono flusso esecuzione, decisioni logiche, ripetizioni.

if, else if, else: Eseguono blocchi in base condizioni specifiche

```
if (x > 5){  
  console.log("Grande");  
} else {  
  console.log("Piccolo");  
}
```

switch: Valuta più valori stessa variabile, alternativa ad if...else

```
switch (colore) {  
  case "rosso":  
    console.log("Stop");  
    break;  
}
```

for: Ripete blocco numero specifico volte

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

for...in: Itera proprietà oggetto

```
for (let key in obj) {  
  console.log(key, obj[key]);  
}
```

for...of: Itera array, valori di strutture iterable

```
for (let val of array) {  
  console.log(val);  
}
```

while: Ripete finché condizione vera

```
while (i < 5) {  
  console.log(i);  
  i++;  
}
```

do...while: Esegue almeno una volta, poi ripete finché condizione vera

```
do {  
  console.log(i);  
  i++;  
} while (i < 5);
```

Operatori

Simboli speciali per operazioni su valori, variabili.

Aritmetici: +, -, *, /, %

Assegnazione: =, +=, -=

Confronto: ==, !=, ===, <, >

Logici: &&, ||, !

Bitwise: &, |, ^, ~

Array

Struttura memorizza sequenza valori (numeri, stringhe, oggetti, altri array). Oggetti con indici numerici (da 0).

Creazione:

```
let array1 = [1, 2, 3, 4, 5];
let array2 = new Array("apple", 1, true, null);
let array3 = new Array(10);
```

Accesso elementi:

```
let el = array[0];
let el = array[array.length - 1];
let el = array[indice] = newVal;
```

Metodi manipolazione:

- .push(el): aggiunge fine array
- .pop(): rimuove/restituisce ultimo elemento
- .unshift(el): aggiunge inizio array
- .shift(): rimuove/restituisce primo elemento

Metodi ricerca:

- .indexOf(el): restituisce indice prima occorrenza (-1 se non presente)
- .includes(el): restituisce true se elemento esiste
- .find(callback): restituisce primo elemento soddisfa condizione

Metodi ordinamento:

- .sort(): ordina alfabetico o personalizzato
- .reverse(): inverte ordine elementi

Metodi iterazione:

- .forEach(callback): esegue funzione per ogni elemento
- .map(callback): crea nuovo array con risultato callback ogni elemento
- .filter(callback): crea nuovo array elementi soddisfano condizione

Oggetti

Collezioni proprietà chiave-valore. Usano indici nominativi.

Creazione notazione letterale:

```
let persona = {
  nome: "Lorena",
  età: 30,
  occupazione: "Insegnante"
};
```

Creazione con costruttore:

```
let persona = new Object();
persona.nome = "Lorena";
persona.età = 30;
persona.occupazione = "Insegnante";
```

Accesso proprietà:

```
let nome = persona["nome"];
let nome = persona.nome;
```

Aggiunta, modifica, rimozione:

```
persona.indirizzo = "Via Roma 1";
persona["email"] = "marco@example.com";
delete persona.occupazione;
```

Metodi: Funzioni associate oggetto

```
let auto = {
  marca: "Fiat",
  modello: "Panda",
  anno: 2020,
  saluta: function() {
    console.log("Ciao, sono una " + this.marca + " " + this.modello);
  }
};
```

this riferimento oggetto stesso, accede proprietà.

DOM (Document Object Model)

Selezione elementi

`document.getElementById("id")`: ritorna elemento con id specifico

`document.getElementsByTagName("tag")`: ritorna tutti elementi con tag sotto forma HTMLCollection

`document.querySelector("selettore")`: seleziona primo elemento corrispondente selettore CSS

`document.querySelectorAll("selettore")`: ritorna tutti elementi corrispondenti selettore, NodeList

Navigazione DOM

Element (solo elementi):

- `children`: figli elementi
- `firstElementChild`: primo figlio
- `lastElementChild`: ultimo figlio
- `nextElementSibling`: fratello successivo
- `previousElementSibling`: fratello precedente
- `parentElement`: genitore

Node (incluso text, commenti):

- `childNodes`: tutti nodi figli
- `firstChild`: primo nodo
- `lastChild`: ultimo nodo
- `nextSibling`: fratello successivo
- `previousSibling`: fratello precedente
- `parentNode`: genitore

Proprietà elementi

textContent: Ottiene/imposta testo HTML. Non interpreta tag

- `element.textContent = "testo"`

className: Ottiene/imposta attributo class. Sovrascrive classi esistenti

- `element.className = "classe-nuova"`

classList: Manipolare classi. Metodi: `add()`, `remove()`, `toggle()`, `contains()`

- `element.classList.add("classe")`
- `element.classList.toggle("classe")`

setAttribute / getAttribute: Imposta/ottiene valore attributo generico

- `element.setAttribute("data-id", "123")`
- `element.getAttribute("data-id")`

value: Legge/imposta valore input, textarea, select

- `input.value = "nuovo valore"`

style: Modifica stile inline elemento (meno performante che CSS)

- `element.style.backgroundColor = "#c00083"`
- `element.style.fontSize = "2em"`

Manipolazione DOM

Creare elementi:

- `document.createElement("tag")`: crea nuovo elemento
- `element.innerHTML = "<p>Contenuto</p>";` inserisce HTML come stringa

Inserire nel DOM:

- `parent.appendChild(node)`: aggiunge come ultimo figlio
- `parent.prepend(node)`: aggiunge come primo figlio
- `parent.insertBefore(newNode, refNode)`: inserisce prima di elemento specifico

Sostituire:

- `parent.replaceChild(newNode, oldNode)`: sostituisce elemento

Rimuovere:

- `element.remove()`: rimuove elemento direttamente
- `parent.removeChild(node)`: rimuove figlio dal genitore
- `element.innerHTML = ""`: svuota figli di elemento

Eventi

Azioni verificano nel browser in risposta interazioni utente.

Aggiungere evento (tre approcci):

1. **HTML inline:** Mescola HTML e JS, un solo gestore evento
 - `<button onclick="alert('Ciao!')">Clicca</button>`
2. **Proprietà JS:** Separa HTML e JS, un solo gestore evento
 - `button.onclick = function() { alert("Ciao!"); };`
3. **Listener:** Più codice, più gestori evento (consigliato)
 - `button.addEventListener("click", function() { alert("Ciao!"); });`

Tipi evento:

- **Mouse:** click (onclick), dblclick (ondblclick), mouseover (onmouseover), ...
- **Tastiera:** keydown (onkeydown), keyup (onkeyup)
- **Form:** submit (onsubmit), change (onchange), focus (onfocus), ...
- **Finestra:** load (onload), resize (onresize), scroll (onscroll), ...

preventDefault(): Impedisce comportamento predefinito evento

```
document.querySelector("form").addEventListener("submit", function(event) {  
    event.preventDefault();  
    console.log("Invio bloccato. Qui puoi gestire i dati.");  
});
```

Esempi: form non ricarica, link non naviga, menu destro bloccato, tasti speciali bloccati

window.onload: aspetta caricamento HTML. Elementi HTML esistono, script li trova e modifica.

```
window.onload = function() {  
    console.log("La pagina e tutte le risorse sono state caricate!");  
};
```

Servizi Google ■ Integrazioni Google per visualizzazioni, mappe, calendari.

Google Charts → Grafici interattivi: PieChart, BarChart, ColumnChart, GeoChart, Gauge, Timeline.

```
google.charts.load('current', {'packages':['corechart']});  
google.charts.setOnLoadCallback(drawChart);
```

```
let data = google.visualization.arrayToDataTable([  
    ['Elemento', 'Valore'],  
    ['A', 10],  
    ['B', 5]  
]);
```

```
let chart = new google.visualization.PieChart(document.getElementById('chart_div'));  
chart.draw(data, {title: 'Titolo'});
```

Eventi: ready, select, error

Google Maps → Mappe interattive, marcatori, percorsi. Tipi: Roadmap, Satellite, Hybrid, Terrain.

Incorporare: google.com/maps → Condividi → Incorpora mappa → copiare iframe

```
<iframe src="https://www.google.com/maps/embed?pb=..." width="800" height="600"></iframe>
```

PRO: Controllo completo JavaScript. **CONTRO:** Richiede API Key.

Google Calendar → Calendario condivisibile con eventi.

Incorporare: calendar.google.com → Impostazioni → Incorpora → copiare iframe

```
<iframe src="https://calendar.google.com/calendar/embed?src=..." width="800" height="600"></iframe>
```

API JS: Codice js chiama API Google Calendar anziché iframe. Richiede chiave API Google Cloud.

6. Node.js

Ambiente esecuzione JavaScript server-side. Runtime basato su Google V8 engine scritto in C++.

Caratteristiche:

- Asincrone e event-driven: gestisce operazioni senza bloccare
- Accesso alle API sistema operativo (file, rete, processi)
- Package manager integrato: NPM
- Usato da Netflix, Uber, PayPal, ...

Node.js (server-side)

- Esecuzione su server
- Accesso file system, rete, database
- No DOM, no Web Storage
- Gestione API, backend

JavaScript (client-side)

- Esecuzione nel browser
- Accesso DOM, Web Storage, API web
- No file system diretto
- Interazioni dinamiche pagina

__dirname: percorso assoluto cartella file corrente.

__filename: percorso assoluto file corrente. Entrambi globali Node.js.

NPM (Node Package Manager)

Gestore ufficiale pacchetti Node.js.

Funzioni:

- Repository online con pacchetti gratuiti
- Installa:
 - pacchetti locali (progetto)
 - pacchetti globali (sistema)
- Gestisce versioni dipendenze
- Esegue script personalizzati

Comandi base

- `npm install <nome_modulo>` → installa locale
- `npm install -g <nome_modulo>` → installa globale
- `npm list` → elenca pacchetti installati
- `npm update <nome_modulo>` → aggiorna
- `npm uninstall <nome_modulo>` → rimuove
- `npm audit` → verifica sicurezza
- `npm prune` → elimina dipendenze non usate

Moduli

Blocco codice isolabile. Organizza, riutilizza, incapsula logica da interferenze.

Node.js: nativi (fs, http) e custom.

Caricamento: `const modulo = require('./modulo');`

Pacchetti

Directory contenente uno o più moduli insieme a file `package.json` con metadati.

package.json: definisce

- Nome, versione, descrizione
- Dipendenze (quali librerie servono)
- Script personalizzati (avviare server, test, ecc.)
- Autore, licenza
- Entry point (file principale)
 - `dependencies`: produzione (Express, database, utility)
 - `devDependencies`: solo sviluppo (nodemon, testing)

package-lock.json: blocca versioni esatte dipendenze.

Garantisce installazioni identiche su tutti sistemi.

Creazione automatica

```
npm init
npm init -y
```

Esempio package.json:

```
{
  "name": "my-project",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "express": "^4.17.1"
  },
  "devDependencies": {
    "nodemon": "^2.0.7"
  },
  "author": "Name",
  "license": "MIT"
}
```

Modulo HTTP

Modulo nativo crea server web. `createServer()` crea server che elabora richieste HTTP. Ogni richiesta passa `req` (dati client) e `res` (risposta da inviare).

```
const http = require('http');
const hostname = '127.0.0.1';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');

  if (req.url === '/') {
    res.end('Home page');
  } else {
    res.statusCode = 404;
    res.end('Non trovato');
  }
});

server.listen(port, hostname, () => {
  console.log(`Server avviato su http://${hostname}:${port}`);
});
```

Express.js

Framework minimalista e flessibile per Node.js. Astrae complessità modulo `http` nativo, semplifica routing, middleware, gestione richieste/risposte.

Installazione: `npm install express`

Inizializzazione e lifecycle:

```
const express = require('express');
const app = express(); // Crea app Express

// 1. Middleware globali (eseguiti su ogni richiesta)
app.use(express.json());

// 2. Routing (associa URL a callback)
app.get('/', (req, res) => {
  res.send('Home');
});

app.post('/api/data', (req, res) => {
  res.json({ messaggio: 'Dati ricevuti' });
});

// 3. Avvia server (ascolta su porta)
app.listen(3000, () => {
  console.log('Server su http://localhost:3000');
});
```

Vantaggi vs modulo `http` nativo:

- Routing intuitivo: `app.get()`, `app.post()`, ecc. vs parsing URL manuale
- Middleware system: processa richieste in catena
- Response helper: `res.json()`, `res.send()`, `res.status()` vs metodi bassi
- Gestione errori centralizzata
- Supporto template engine nativo

Caratteristiche chiave:

- **Routing:** `app.METHOD(PATH, CALLBACK)` mappa URL a handler
- **Middleware:** funzioni intercettano richieste prima rotta finale (logging, auth, validazione)
- **Template engine:** EJS, Pug per HTML dinamico server-side
- **Static files:** `express.static()` serve CSS, JS, immagini

Express: Routing

Associa URL e metodi HTTP a funzioni callback.

Ordine rotte: specifiche prima generiche.

Sintassi: `app.METHOD(PATH, CALLBACK)`

Metodi HTTP

- `app.get()`: leggi dati
- `app.post()`: crea risorse
- `app.put()`: modifica risorse
- `app.delete()`: elimina risorse

Status HTTP

- 200 OK, 201 Created
- 404 Not Found, 400 Bad Request
- 500 Server Error

Wildcard: `app.get('*')` → catch-all

URL params (obbligatori, parte percorso)

```
app.get('/greet/:name', (req, res) => {  
  const name = req.params.name;  
  res.send(`Ciao, ${name}!`);  
});
```

Accesso: `/greet/Marco`

Query string (opzionali, dopo ?)

```
app.get('/greet', (req, res) => {  
  const name = req.query.name || 'Ospite';  
  res.send(`Ciao, ${name}!`);  
});
```

Accesso: `/greet?name=Marco`

Express: Metodi e proprietà

Response – Metodi inviare risposta:

- `res.status(code)`: imposta codice stato HTTP
- `res.send(body)`: invia risposta testo
- `res.json(obj)`: invia risposta JSON
- `res.sendFile(path)`: invia file
- `res.on(event, callback)`: ascolta eventi risposta
- `res.cookie(name, value)`: imposta cookie
- `res.clearCookie(name)`: rimuove cookie

Request – Proprietà richiesta HTTP:

- `req.params`: parametri URL
- `req.query`: parametri query string
- `req.body`: corpo richiesta HTTP
- `req.headers`: header richiesta
- `req.cookies`: cookie inviati da client
- `req.method`: metodo HTTP della richiesta
- `req.ip`: indirizzo IP del client

Express: Middleware

Funzione che intercetta richieste HTTP prima destinazione finale.

Accesso a:

- `req`: oggetto richiesta
- `res`: oggetto risposta
- `next()`: passa controllo middleware successivo

Esecuzione: in ordine dichiarazione prima delle rotte.

Tipi middleware:

Built-in (già inclusi Express):

- `express.static()`: servire file statici
- `express.json()`: elaborare corpo JSON
- `express.urlencoded()`: elaborare form dati

Custom (scritti manualmente):

```
app.use((req, res, next) => {  
  // Logica middleware  
  next(); // Passa al prossimo  
});
```

Third-party (moduli esterni come morgan, cors, ecc.):

```
const morgan = require('morgan');  
app.use(morgan('combined'));
```

AJAX (Asynchronous JavaScript And XML)

Tecnica per inviare/ricevere dati server async senza bloccare/refresh. JSON sostituisce XML.

Architetture

1. Server-side rendering (tradizionale)

- Server genera HTML completo, client visualizza
- ✓ Funziona senza JS, meno richieste
- ✗ Ogni azione richiede reload

2. Client-side (AJAX)

- Server invia JSON, client aggiorna DOM dinamicamente
- ✓ Update parziali, no reload, fluido
- ✗ Dipende JS, gestione backend/frontend più complessa

Flusso

1. Utente clicca → JS invia fetch asincrono
2. Server riceve, elabora DB, invia JSON
3. JS riceve, aggiorna DOM, no refresh

Fetch API

API moderna per richieste HTTP. Ritorna **Promise** (oggetto risultato futuro: pending → fulfilled/rejected).

async: dichiara funzione asincrona che ritorna Promise. Permette l'uso di `await` dentro.

await: sospende finché Promise risolto. Non blocca programma, syntactic sugar per `.then()`.

```
const res = await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(dati)
});
const data = await res.json();
```

Esempio completo con error handling:

```
async function load() {
  try {
    const res = await fetch("/api/dati");
    if (!res.ok) throw new Error("HTTP error");
    return await res.json();
  } catch (err) { console.error(err); }
}
```

Parametri:

- method:
 - GET: leggi dati da server
 - POST: crea nuova risorsa
 - PUT: modifica risorsa esistente
 - DELETE: elimina risorsa
- headers: "Content-Type": "application/json" dichiara tipo dati
- body: Stringa JSON con `JSON.stringify()`

express.json() Middleware:

Legge body HTTP JSON, carica in `req.body`.

Alternativa: `express.urlencoded()` per form HTML.

```
app.use(express.json());
app.post("/api/submit", (req, res) => {
  const { nome } = req.body;
  res.json({ messaggio: "OK" });
});
```

Virtual DOM & Framework

React/Vue usano Virtual DOM per ottimizzare updates. Overhead spesso supera benefici per app semplici.

Best Practice

- Siti statici/serverless quando possibile
- Vanilla JS/HTML/CSS senza framework pesanti
- No librerie per feature minime
- Lazy loading immagini, ridurre peso pagine
- No tracking/analytics superflui

Casi Reali

GitHub: Rimosso jQuery, usa metodi nativi browser.

Netflix: Eliminato React dal frontend, JavaScript vanilla, riduzione 50% load time.

Airbnb: Lazy loading immagini intelligente, riduzione significativa peso pagine.

7. Database

Connessione e gestione dati lato server. Node.js supporta MySQL, PostgreSQL, MongoDB tramite librerie.

Tre approcci: isolata, condivisa, pool.

Connessione isolata

- `createConnection()` per ogni route
- Vantaggio: sicurezza concorrenza
- Svantaggio: costi overhead apertura/chiusura per ogni richiesta

Connessione condivisa

- `createConnection()` globale
- Vantaggio: solo apparente
- Svantaggio: problemi stessi della connessione globale

Connection Pool

Insieme connessioni riutilizzate. Pool gestisce:

- Richieste concorrenti
- Riutilizzo efficiente connessioni
- Rilascio automatico fine query

Vantaggi: evita overhead connessioni singole, mantiene performance, scalabilità.

Creazione pool

```
const pool = mysql.createPool({
  host: "localhost",
  user: "user1",
  password: "pass1",
  database: "mio_db",
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
```

Configurazione

- `createPool()`: crea pool connessioni DB
- `waitForConnections`: aspetta connessione se pool pieno
- `connectionLimit`: max connessioni simultanee
- `queueLimit`: max richieste in attesa (0 = illimitato)
- `.promise()`: abilita `async/await` con `.execute()`
- `.release()`: rilascia connessione al pool

Prepared Statement

Query parametrizzata. Placeholder (?) separano struttura SQL dai dati. Server elabora: prima compila struttura, poi inserisce valori come dati (non codice).

Perché: previene SQL injection. Dati non interpretati come codice SQL.

Insicuro: `const query = "INSERT INTO utenti VALUES ('" + nome + "', '" + email + "')";`
`connection.query(query);` Se `nome = ''`; `DROP TABLE--` esegue DROP.

Sicuro: `const query = "INSERT INTO utenti VALUES (?, ?)";`
`connection.execute(query, [nome, email]);`

Valori sostituiti come dati, mai interpretati come SQL.

Uso Pool

```
app.get("/users", async (req, res) => {
  const connection = await pool.promise().getConnection();
  try {
    const [rows, cols] = await connection.execute("SELECT * FROM utenti");
    res.json(rows);
  } catch (err) {
    res.status(500).json({success: false, message: "Errore"});
  } finally {
    connection.release();
  }
});
```

Transactions

Unità atomica: raggruppa operazioni DB in singola operazione logica. Garanzia: tutte eseguono con successo OPPURE nessuna applicata. Niente stato intermedio.

Perché serve: evita inconsistenza dati. Se operazione 1 di 3 fallisce, altre 2 non lasciano dati corrotti.

Garantisce: consistenza. Operazione fallisce = rollback automatico cancella tutte le modifiche.

Esempio:

trasferimento denaro fra conti

1. Addebito primo conto
2. Accredito secondo conto

Se una fallisce, nessuna applicata.

Metodi transazione

- `beginTransaction()`: avvia transazione asincrona
- `execute()`: esegue singola query
- `commit()`: rende permanenti modifiche
- `rollback()`: annulla modifiche se errore
- `release()`: libera connessione al pool

Esempio Transazione

```
app.post("/transfer", async (req, res) => {
  const { from_account, to_account, amount } = req.body;
  const conn = await pool.promise().getConnection();
  try {
    await conn.beginTransaction();

    const debit = "UPDATE accounts SET balance = balance - ? WHERE id = ?";
    await conn.execute(debit, [amount, from_account]);

    const deposit = "UPDATE accounts SET balance = balance + ? WHERE id = ?";
    await conn.execute(deposit, [amount, to_account]);

    await conn.commit();
    res.json({ success: true, message: "Transfer OK" });
  } catch (err) {
    await conn.rollback();
    res.status(500).json({ success: false, message: "Transfer error" });
  } finally {
    conn.release();
  }
});
```

8. Autenticazione

Gestione login e sessioni. JWT (JSON Web Token) standard per autenticazione stateless.

Cookie

Dove: client (browser dati)

Accesso: browser legge, server scrive e legge cookie

Durata: persistente o scadenza definita

Rischio: dati visibili, modificabili se non firmati

Sessione

Dove: server (RAM/DB)

Accesso: server gestisce, cookie ha solo ID

Durata: temporanea, scade se logout/inattività

Rischio: dati protetti, server controlla tutto

Per il Login

Sessioni: Server crea ID, salva dati lato server, cookie contiene solo ID.

Cookies: Server salva dati nel cookie stesso. Dati visibili a chiunque.

JWT (JSON Web Token)

Token compatto, firmato e auto-contenuto. Tre parti separate da punti: HEADER.PAYLOAD.SIGNATURE.

Esempio token reale:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQ0jEYMywiZW1haWwiOiJlc2VyMUBlbWFpC5jb20ifQ...

Tutto codificato base64url (URL-safe).

JWT Header

Descrive come è firmato il token:

```
{ "alg": "HS256", "typ": "JWT" }
```

- alg: algoritmo firma (HS256, RS256)
- typ: tipo token

JWT Payload

Contiene informazioni da trasmettere (claims):

```
{ "userId": 123, "role": "admin", "exp": 1716103600 }
```

- userId, role: custom claims (dati app)
- exp: timestamp scadenza

JWT Signature

Verifica che token non sia stato modificato. Calcolata così:

```
HMACSHA256(base64(header) + "." + base64(payload), JWT_SECRET)
```

- Server firma token con chiave segreta
- Se token viene modificato, firma non combacia
- Se token scaduto (exp < now), è invalido

Creazione JWT

```
const jwt = require("jsonwebtoken");
const JWT_SECRET = "mia_chiave_super_segreta";

const payload = { userId: "user1", email: "user1@email.com" };
const token = jwt.sign(payload, JWT_SECRET, {
  algorithm: "HS256",
  expiresIn: "1h"
});
```

Avvertenza: non inserire dati sensibili nel payload. È visibile a chiunque legga il token.

Login

```
app.post("/api/login", async (req, res) => {
  const { email, password } = req.body;
  const query = "SELECT id, nome FROM utenti WHERE email = ? AND password = ?";

  try {
    const [rows] = await pool.promise().execute(query, [email, password]);
    const user = rows[0];

    const payload = { userId: user.id, userName: user.nome };
    const token = jwt.sign(payload, JWT_SECRET, {
      algorithm: "HS256",
      expiresIn: "1h"
    });

    res.cookie("token", token, {
      httpOnly: true,
      secure: true,
      maxAge: 3600000,
      sameSite: "Strict"
    });

    return res.json({ success: true, message: "Login riuscito" });
  } catch (err) {
    res.status(500).json({ success: false, message: "Errore" });
  }
});
```

Passi:

1. Verifica credenziali in DB
2. Prepara payload con userId, ruolo
3. `jwt.sign()` genera token firmato
4. Imposta cookie con token (`httpOnly`, `secure`, `sameSite`)

Middleware - Pagine Protette

```
function authenticateToken(req, res, next) {  
  const token = req.cookies.token;  
  if (!token) {  
    return res.redirect(302, "/index.html");  
  }  
  
  try {  
    const payload = jwt.verify(token, JWT_SECRET);  
    req.user = payload;  
    next();  
  } catch (err) {  
    return res.redirect(302, "/index.html");  
  }  
}  
  
app.use("/private", authenticateToken, express.static("private"));
```

Flusso:

- Estrai token da cookie
- Se manca/invalido → redirect
- Se valido → salva payload, continua

Logout

```
app.post("/api/logout", authenticateToken, (req, res) => {  
  res.clearCookie("token");  
  res.json({ success: true, message: "Logout effettuato" });  
});
```

`clearCookie()` rimuove il token dal browser.

Problemi di sicurezza comuni

1. **XSS (Cross-Site Scripting):** attaccante inietta JS che ruba token da `localStorage`
 - Usa `httpOnly` cookies (JS non accede)
 - Usa `Secure` per HTTPS solo
 - Sanitizza sempre input/output HTML
 - Content-Security-Policy lato server
2. **CSRF (Cross-Site Request Forgery):** sito esterno induce browser a fare richieste non volute
 - `SameSite=Lax` o `Strict` nei cookie
 - Token CSRF per form
3. **Token Replay:** token rubato viene riutilizzato
 - Scadenza breve (`expiresIn: «1h»`)
 - Blacklist token su logout
4. **Token Leakage:** token finisce in log, URL, ecc.
 - Non passare JWT in query string
 - Usa `Authorization` header oppure `httpOnly` cookie
5. **Manomissione Chiave:** chiave debole/compromessa
 - Lunghe e segrete per HS256
 - RS256 con chiave privata (distribuito)

9. Computazioni Pesanti

Task pesante (parsing, ML, encoding, calcoli) bloccano main thread → UI congela, input non risponde.

Problema: JavaScript single-threaded = browser inutilizzabile finché non termina.

Due approcci con trade-off diversi:

Lato client

- Niente latenza rete
- Carico distribuito
- Dati rimangono locale
- Esempi:
 - Compressione immagini, parsing file, crittografia
 - Rendering canvas (grafica, giochi)
 - ML in browser (TensorFlow.js)

Lato server

- Client leggero (poca potenza)
- Logica centralizzata (DB, filesystem, API)
- Computazioni con GPU/CPU dedicate
- Esempi:
 - ML lato server
 - Big data analytics
 - Video encoding
 - Logica business riservata

Web Workers (Browser)

Esecuzione JS in thread separato: main thread gestisce DOM/UI, worker gestisce calcoli senza bloccare. Richiedono server HTTP.

Architettura

Comunicazione bidirezionale tramite messaggi (no shared memory).

```
// Main Thread (script.js)
// Crea worker e invia dati serializzabili
const worker = new Worker("script/worker.js");
worker.postMessage({ max: 20000000 });

// Riceve messaggi dal worker (in event.data)
worker.onmessage = (event) => {
  if (event.data.type === "progress") {
    progress.value = event.data.value;
  }

  else if (event.data.type === "done") {
    progress.value = 100;
    resP.textContent = "Numeri primi trovati: " + event.data.count;
    worker.terminate(); // Termina il worker
  }
};

// Worker Thread (script/worker.js)
// Riceve dati dal main thread
onmessage = function (e) {
  const max = e.data.max;
  const primes = [];

  for (let i = 0; i < max; i++) {
    if (isPrime(i)) primes.push(i);

    if (i % 100000 === 0) {
      postMessage({ type: "progress", value: (i / max) * 100 });
    }
  }

  // Invia risultati al main thread
  postMessage({ type: "done", count: primes.length });
};
```

Interrompere il worker

- Chiusura scheda → termina main thread e worker
- worker.terminate() dal main
- self.close() dal worker
- Timeout con setTimeout()
- Non riprendibile dopo pause

Worker Threads (Node.js)

Modulo nativo per thread paralleli senza bloccare event loop. Main thread crea Workers che eseguono JS separato, comunicazione via `postMessage` / `on("message")`.

Configurazione

```
const { Worker, isMainThread, parentPort, workerData } = require("worker_threads");
```

- **Worker:** crea thread con file JS
- **isMainThread:** verifica contesto (main vs worker)
- **parentPort:** porta comunicazione main↔worker
- **workerData:** dati passati all'avvio del worker

Esempio: calcolo numeri primi (Server.js)

```
const { Worker } = require("worker_threads");
...
app.get("/calcola", (req, res) => {
  const n = parseInt(req.query.count);
  const worker = new Worker("./worker.js", { workerData: n });

  req.on("close", () => {
    console.log("Client disconnesso, termino il worker");
    worker.terminate();
  });
  worker.on("message", primes => {
    res.json({ primes });
  });
  worker.on("error", err => {
    console.error("Errore nel worker:", err);
    res.status(500).send("Errore nel worker: " + err.message);
  });
});
...

```

Server-side Techniques

Comparazione metodi trasmissione risultati pesanti.

Tecnica	Comunicazione	Progressivo?	Interrompibile?	Difficoltà	Note
Fetch	Req/Resp HTTP	No	No	Bassa	Semplice
Polling	Richieste ripetute	Sì	Sì	Media	Più richieste
SSE	HTTP unidirez.	Sì	Parziale	Media	Push server→client
WebSocket	Full-duplex	Sì	Sì	Alta	Real-time bidirez.
ReadableStream	Streaming HTTP	Sì	No	Alta	Chunk dati